

Tareas GForma

Sistema de gestión de tareas, tiempos y presencia

Máximo Dabarca Maneiro

2.º DAM · Nortempo Formación · Junio 2026

Índice

1. Motivación e introducción

- 1.1 Por qué este proyecto
- 1.2 El problema que resuelve
- 1.3 Objetivos

2. Metodología, recursos y planificación

- 2.1 Cómo trabajé
- 2.2 Qué necesité
- 2.3 Planificación

3. Tecnologías y herramientas

4. Análisis

- 4.1 Requisitos
- 4.2 Modelo de datos
- 4.3 Casos de uso principales

5. Diseño

- 5.1 Arquitectura
- 5.2 Diagrama de clases
- 5.3 Pantallas principales
- 5.4 Decisiones de diseño que tomé

6. Implementación

- 6.1 Estructura del proyecto
- 6.2 Componentes principales
- 6.3 Problemas que encontré y cómo los resolví
- 6.4 Consultas no triviales

7. Multiusuario concurrente

- 7.1 Cómo se identifican los usuarios
- 7.2 Prueba con dos usuarios simultáneos
- 7.3 Gestión de fallos de comunicación
- 7.4 Evidencia de operación

8. Despliegue e instalación

- 8.1 Instalación
- 8.2 Configuración
- 8.3 Datos de ejemplo
- 8.4 Manual de usuario (breve)

9. Pruebas

- 9.1 Plan de pruebas
- 9.2 Casos de prueba

10. Resultados, conclusiones y vías futuras

- 10.1 Qué conseguí
- 10.2 Conclusiones + lecciones aprendidas

- 10.3 Por dónde seguir
- 10.4 Reflexión personal
- 10.5 Métricas del proyecto
- 10.6 Tabla resumen

11. Glosario

12. Bibliografía

Anexos

- Anexo A – Capturas adicionales
- Anexo B – Evidencias de pruebas

Resumen

Tareas GForma es una aplicación de escritorio hecha con Electrón que surgió de una necesidad bastante concreta: tener todo en un solo sitio. Antes de desarrollarla, la información del equipo estaba repartida entre hojas de cálculo, correos sueltos y algún que otro documento que nadie sabía dónde estaba. Eso complicaba el seguimiento de tareas, los horarios y cualquier cosa que implicara buscar algo de hace dos semanas.

La aplicación centraliza clientes, tareas, presupuestos, tiempos de trabajo y control de presencia, conectándose a una base de datos SQL Server compartida. Así varias personas pueden trabajar a la vez sin perder nada por el camino.

Las cosas que hace la aplicación, resumidas:

- Inicio de sesión con validación de usuario y contraseña.
- Gestión de clientes, tareas, contactos y presupuestos.
- Control de presencia y consulta de horarios.
- Registro de tiempos asociados a tareas.
- Configuración de la conexión a SQL Server sin dejar credenciales a la vista.

1. Motivación e introducción

1.1 Por qué este proyecto

La idea de Tareas GForma viene de haber visto cómo se gestiona el trabajo en muchas oficinas pequeñas: con Excel, con WhatsApp y con mucha memoria. Eso funciona hasta cierto punto, pero en cuanto el equipo crece un poco o hay que auditar algo, se complica bastante.

Quería hacer algo que de verdad se pudiera usar, no solo un ejercicio académico. Por eso el proyecto tiene base de datos real, autenticación, módulos concretos y pruebas. Es el tipo de aplicación que cualquier empresa pequeña podría necesitar.

1.2 El problema que resuelve

Sin una herramienta así, los datos suelen vivir en sitios distintos: tareas en un Excel, presencia apuntada a mano, presupuestos en otro documento... Tareas GForma mete todo eso en una sola base de datos y ofrece una interfaz clara para consultarlo y modificarlo. Nada del otro mundo, pero se nota la diferencia.

1.3 Objetivos

En general, lo que quería conseguir era:

- Tener una aplicación de escritorio funcional de verdad.
- Centralizar la información en una base de datos relacional.

Más en concreto:

- Autenticación de usuarios con control de sesión.
- CRUD completo para clientes, tareas, contactos y presupuestos.
- Módulo de tiempos y de presencia.

- Interfaz navegable y que no sea un dolor de cabeza usar.
- Configuración de conexión sin credenciales hardcodeadas.
- Pruebas unitarias por módulos.

2. Metodología, recursos y planificación

2.1 Cómo trabajé

Fui haciendo el proyecto por bloques. Primero la conexión a la base de datos, luego la autenticación, después los modelos y controladores, y al final las vistas y las pruebas. Así podía probar cada parte antes de pasar a la siguiente, sin que un error en un módulo me bloqueara todo lo demás.

Es una metodología iterativa de manual, pero funciona bien para proyectos en solitario donde no hay un equipo que te espere en cada sprint.

2.2 Qué necesité

Hardware:

- Un equipo Windows para desarrollar.
- Un servidor con SQL Server para la base de datos.
- Otro equipo para pruebas de escritorio.

Software:

- Node.js y Electrón.
- SQL Server + SQL Server Management Studio.
- Visual Studio Code y Git/GitHub.
- Jest para las pruebas.

2.3 Planificación

Dividí el trabajo en cuatro fases:

1. Análisis del problema, diseño de datos y configuración del entorno.
2. Implementación de la conexión a SQL Server y autenticación.
3. Desarrollo de modelos, controladores y vistas.
4. Pruebas, correcciones y documentación.

Fase	Descripción	Hito	Fecha
Fase 1	Análisis, diseño de datos y configuración del entorno	Entorno configurado y base de datos creada	Marzo 2026
Fase 2	Conexión a SQL Server y autenticación	Login operativo y conexión funcionando	Abril 2026

Fase 3	Desarrollo de modelos, controladores y vistas	Todos los módulos operativos	Mayo 2026
Fase 4	Pruebas, correcciones y documentación	App empaquetada y memoria entregada	Mayo/Junio 2026

3. Tecnologías y herramientas

Las tecnologías que usé y por qué:

Tecnología / Herramienta	Uso en el proyecto
JavaScript	Lenguaje principal tanto en el proceso main como en las vistas.
HTML / CSS	Maquetación de todas las pantallas de la aplicación.
Electrón	Framework que permite empaquetar la app como escritorio.
SQL Server	Base de datos relacional donde se guarda toda la información.
mssql	Paquete Node.js para conectar con SQL Server.
Jest	Pruebas unitarias de los modelos.
Git / GitHub	Control de versiones durante todo el desarrollo.

4. Análisis

4.1 Requisitos

Requisitos funcionales:

- Inicio de sesión con usuario, contraseña y opcionalmente usuario de Windows.
- CRUD de clientes, contactos, tareas y presupuestos.
- Registro y consulta de tiempos de trabajo por tarea.
- Registro de fichajes y generación de informes de presencia.
- Configuración de la conexión a la base de datos desde la propia app.

Requisitos no funcionales:

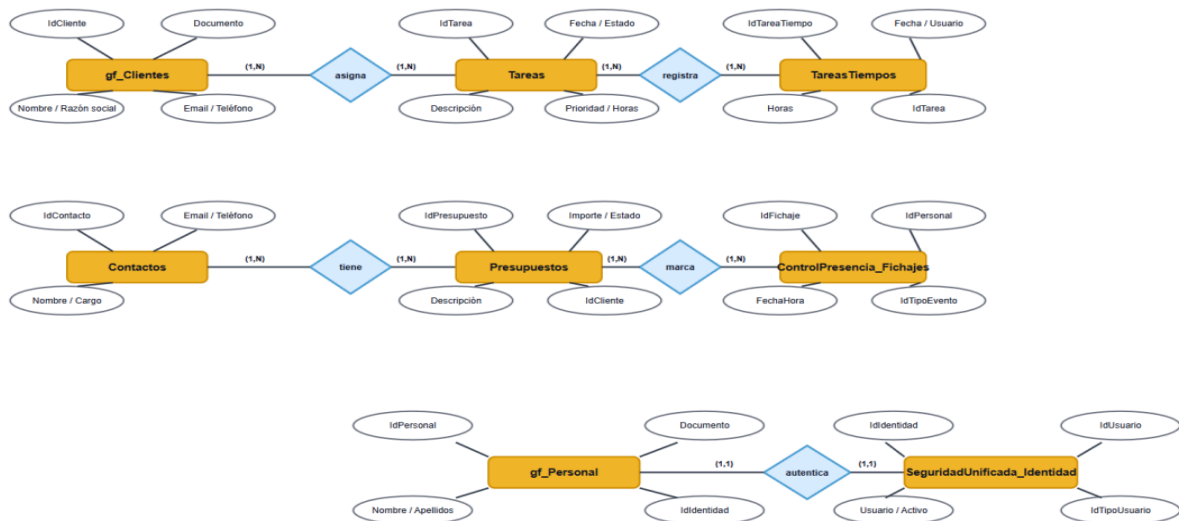
- Aplicación de escritorio, no web.
- Integridad y consistencia de datos garantizadas por SQL Server.
- Sin credenciales en el repositorio.
- Interfaz usable sin formación previa.
- Soporte de acceso concurrente de varios usuarios.
- Código modular y con cobertura de pruebas.

4.2 Modelo de datos

Elegí SQL Server porque el tipo de datos que maneja la aplicación encaja muy bien con un modelo relacional: hay relaciones claras entre entidades, necesito integridad referencial y las consultas son complejas. No tenía sentido usar otra cosa.

Las tablas principales son:

- **gf_Personal**: personal del sistema y su vinculación con el usuario autenticado.
- **gf_Clientes**: datos de clientes.
- **Tareas**: tareas con su estado y prioridad.
- **TareasTiempos**: imputaciones de horas a tareas.
- **ControlPresencia_Fichajes**: fichajes de entrada, salida y pausas.
- **Presupuestos**: presupuestos asociados a clientes.
- **Contactos**: personas de contacto de cada cliente.



4.3 Casos de uso principales

1. El usuario inicia sesión con sus credenciales.
2. El usuario accede a la pantalla principal tras autenticarse.
3. El usuario registra su fichaje de entrada o salida
4. El usuario crea o edita una tarea.
5. El empleado imputa horas a una tarea.
6. El responsable consulta el informe de presencia por fechas.
7. El usuario accede al módulo de presupuestos.
8. Alguien cambia la configuración de conexión si el servidor cambia.
9. El sistema registra acciones y responde con mensajes de estado.

5. Diseño

5.1 Arquitectura

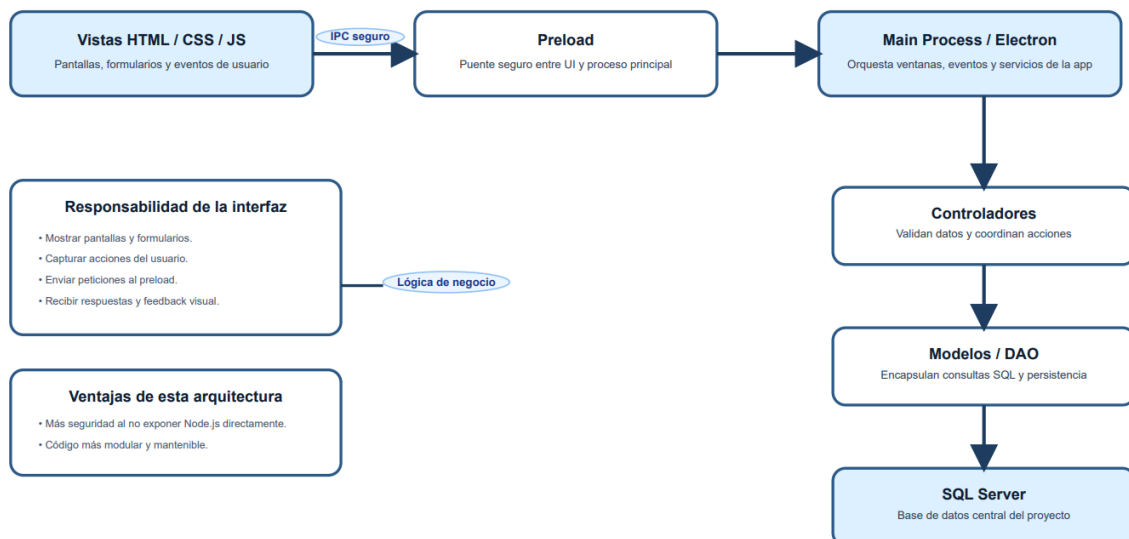
La aplicación sigue la arquitectura típica de Electrón, separando el proceso principal del proceso de renderizado. Hay cuatro capas:

- Vistas (HTML/CSS/JS): lo que ve el usuario, formularios y navegación.
- Preload: puente seguro que expone sólo las funciones necesarias al renderer.
- Proceso principal + controladores: lógica de negocio y coordinación de acciones.
- Modelos / DAO: consultas SQL encapsuladas, sin lógica de presentación.

Esta separación tiene ventajas claras: la interfaz no tiene acceso directo a Node.js (más seguridad), y cada capa se puede probar y mantener por separado. Cuando algo falla, es más fácil saber dónde está el problema.

Diseño general / arquitectura

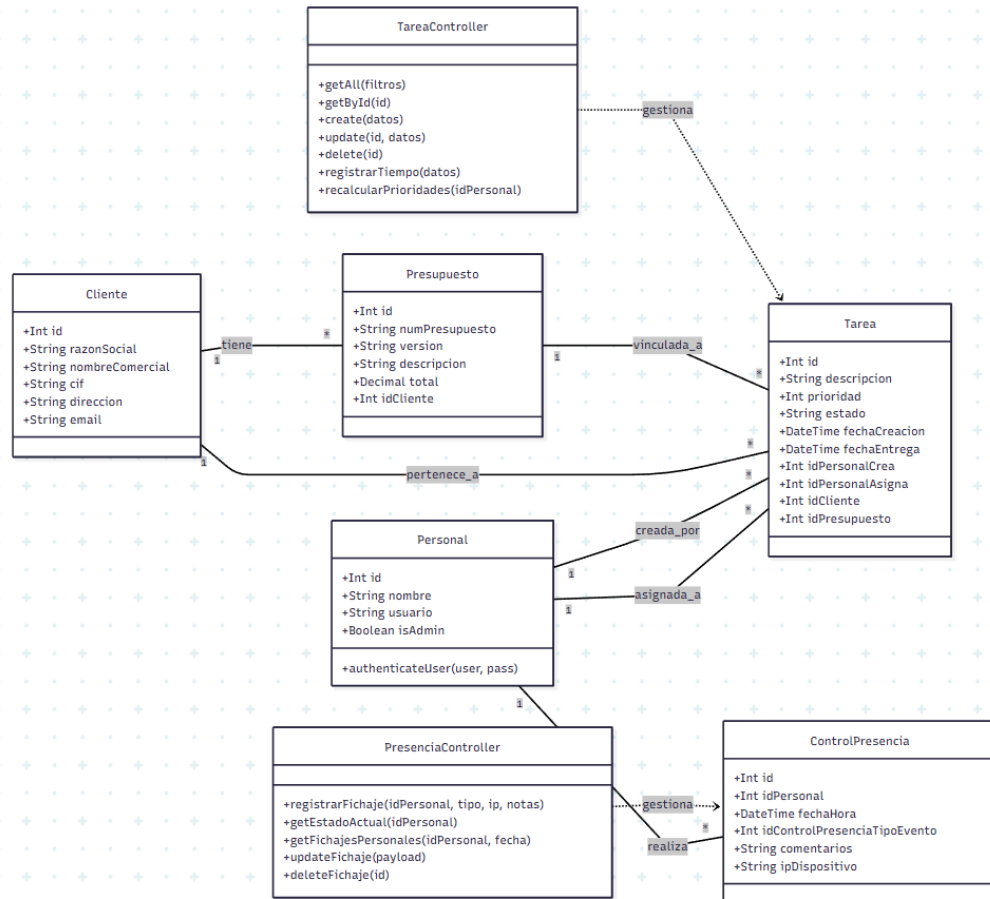
Separación entre interfaz, puente seguro, lógica de aplicación y acceso a datos



5.2 Diagrama de clases

Las clases principales del sistema:

- **db.js**: gestión de la conexión, autenticación y configuración.
- **ClienteModel**, **TareaModel**, **PersonalModel**, **PresenciaModel**, **PresupuestoModel**, **ContactoModel**: modelos de acceso a datos.
- **ClienteController**, **TareaController**, **PresenciaController**, **PresupuestoController**, **ContactoController**: coordinación entre UI y modelos.



5.3 Pantallas principales

- Pantalla de conexión: para configurar o seleccionar el servidor SQL.
- Pantalla de login: autenticación con usuario y contraseña.
- Dashboard principal: lista de tareas activas con filtros por estado.
- Pantalla de crear tarea y modificar tarea.
- Pantalla de ver tiempos.
- Pantalla de Horarios.
- Informe presencia.

5.4 Decisiones de diseño que tomé

Las cuatro decisiones que más condicionaron el resultado:

1. Separar los modelos de acceso a datos en archivos independientes. Hace el código más limpio y cada modelo es fácilmente testeable.
2. Usar IPC entre el renderer y el proceso principal en lugar de dar acceso directo a Node.js desde las vistas. Es la forma segura de hacerlo en Electrón.
3. Externalizar la configuración de base de datos en Config.xml y variables de entorno. Así las credenciales no viajan en el repositorio.
4. Vistas HTML separadas por funcionalidad. Más archivos, pero cada pantalla es independiente y fácil de localizar.

6. Implementación

6.1 Estructura del proyecto

La estructura principal del repositorio:

main.js → arranque de Electrón y ventana principal

preload.js → puente seguro UI ↔ proceso principal

src/database/db.js → conexión, autenticación, configuración

src/models/ → modelos de acceso a datos

src/controllers/ → lógica de negocio

src/ipc/ipcHandlers.js → registro de manejadores IPC

src/views/ → pantallas HTML/CSS/JS

scripts/ → utilidades y pruebas auxiliares

docs/ → documentación y memoria

6.2 Componentes principales

Cómo está organizado cada bloque:

- **Persistencia:** los modelos hacen las consultas a SQL Server usando el paquete mssql. Cada entidad tiene su propio archivo.
- **Controladores:** reciben las peticiones del renderer, validan los datos y coordinan con los modelos.
- **Autenticación:** db.js centraliza la validación de credenciales y la gestión de la sesión activa.
- **Interfaz:** vistas HTML con sus propios scripts que manejan formularios, tablas y navegación.
- **IPC:** ipcHandlers.js registra todos los canales de comunicación entre renderer y proceso principal.

6.3 Problemas que encontré y cómo los resolví

Tres problemas concretos que me dieron trabajo:

Credenciales en el repositorio.

Al principio tenía la cadena de conexión directamente en el código, lo cual es un problema obvio. La solución fue usar un archivo .env junto con Config.xml para externalizar todo eso. El archivo .env está en .gitignore y nunca sube al repositorio.

Mezclar lógica de interfaz y de datos.

En las primeras versiones, algunas vistas hacían consultas directamente. Lo separé aplicando la arquitectura de modelos y controladores de forma consistente. Costó un poco de refactorización pero valió la pena.

Consistencia en fechas y registros de presencia.

Los fichajes son sensibles a la hora exacta y al formato de fecha. Normalicé todo en la capa de datos para que siempre se guarde y se lea en el mismo formato, independientemente de la configuración regional del equipo.

6.4 Consultas no triviales

Consulta 1: Informe de presencia y auditoría con aislamiento de datos

Esta consulta es la encargada de generar el historial de modificaciones y eliminaciones de registros de jornada, garantizando la integridad referencial incluso cuando el dato original ha desaparecido.

- **Qué hace:** Realiza un LEFT JOIN entre la tabla de auditoría (ControlPresencia_Fichajes_Logs) y la tabla de fichajes activos. Utiliza funciones de filtrado temporal (DATEADD, ISNULL) para capturar registros en un rango de fechas exacto y emplea una lógica de "seguridad por ID" para asegurar que solo se recuperen los logs pertenecientes al empleado consultado, tratando de forma especial los registros de tipo DELETE.
- **Por qué es útil:** Es fundamental para el cumplimiento de la normativa laboral y el control administrativo. Permite reconstruir qué pasó con un registro (quién lo cambió, cuándo y por qué) aunque este haya sido borrado por error, manteniendo un historial inalterable por empleado.
- **Evidencia:**

```
const query = `
SELECT
    1.IdControlPresenciaFichajeLog,
    1.IdControlPresenciaFichaje,
    1.FechaHoraModificacion,
    1.UsuarioModificacion,
    1.Accion,
    1.CampoModificado,
    1.ValorAnterior,
    1.ValorNuevo,
    1.Motivo
FROM dbo.ControlPresencia_Fichajes_Logs 1
LEFT JOIN dbo.ControlPresencia_Fichajes f ON f.IdControlPresenciaFichaje = 1.IdControlPresenciaFichaje
WHERE (@Desde IS NULL OR 1.FechaHoraModificacion >= @Desde)
    AND (@Hasta IS NULL OR 1.FechaHoraModificacion < DATEADD(DAY, 1, @Hasta))
    AND (@IdPersonal IS NULL OR (1.IdPersonal = @IdPersonal) OR (1.IdPersonal IS NULL AND f.IdPersonal = @IdPersonal))
ORDER BY 1.FechaHoraModificacion DESC, 1.IdControlPresenciaFichajeLog DESC
`;
```

Consulta 2: Listado de tareas con priorización dinámica y cruce de entidades

Es la consulta principal del módulo de trabajo, encargada de proyectar una visión clara de la carga laboral actual.

- **Qué hace:** Ejecuta un procedimiento almacenado (up_bp_Tareas_Select) que realiza múltiples INNER JOIN entre las tablas de Tareas, gf_Clientes, TareasTipos y SeguridadUnificada_Identidad. Aplica una ordenación jerárquica: primero por estado (agrupando las activas), luego por el número de prioridad calculado y finalmente por cliente. Además, realiza un cálculo acumulativo de las horas invertidas mediante una subconsulta.
- **Por qué es útil:** Facilita la toma de decisiones al responsable de equipo. Al unificar en una sola vista el nombre del cliente, el tipo de tarea y el tiempo real que se está dedicando, el administrador puede identificar cuellos de botella y ajustar las prioridades (que el sistema reordena automáticamente) para optimizar la productividad del personal asignado.

- **Evidencia:**

```

SELECT
    Tar.IdTarea,
    Tar.IdPersonal_Asigna AS IdPersonalAsignado,
    Tar.Descripcion,
    Tar.Prioridad,
    ISNULL(Tiem.Total,0) AS HorasReales,
    CASE WHEN Tar.FComprobada IS NULL AND Tar.FRealizada IS NULL AND Tar.FechaEnEspera IS NOT NULL THEN 5 -- En Espera
         WHEN Tar.FComprobada IS NOT NULL THEN 4 -- Comprobada
         WHEN Tar.FRealizada IS NOT NULL THEN 3 -- Realizada
         WHEN Tar.FIniciada IS NOT NULL THEN 2 -- Iniciada
         ELSE 1 -- Asignada
    END AS IdTareaEstado,
    -- ... [Aquí vienen el resto de campos] ...
FROM
    dbo.Tareas Tar
INNER JOIN
    dbo.gf_Personal PerCre ON Tar.IdPersonal_Crea = PerCre.IdPersonal
INNER JOIN
    dbo.gf_Personal PerAsi ON Tar.IdPersonal_Asigna = PerAsi.IdPersonal
INNER JOIN
    dbo.gf_Clientes Cli ON Cli.IdCliente = Tar.IdCliente
INNER JOIN
    dbo.TareasTipos Tip ON Tip.IdTareaTipo = Tar.IdTareaTipo
LEFT JOIN
    dbo.vw_Tareas_Tiempos Tiem ON Tiem.IdTarea = Tar.IdTarea

```

7. Multiusuario concurrente

7.1 Cómo se identifican los usuarios

La identificación de usuarios se realiza mediante un sistema de Autenticación Basada en Sesión:

- **Login:** Al iniciar la aplicación, el usuario debe introducir sus credenciales. El sistema valida el usuario y la contraseña contra la tabla *dbo.SeguridadUnificada_Identidad*.
- **Sesión:** Una vez autenticado, el servidor devuelve un objeto de sesión que se mantiene en la memoria del proceso principal (Main Process) de Electrón. Esta sesión contiene el idPersonal, el nombre del usuario y su rol (isAdmin).
- **Seguridad:** Cada petición al controlador (IPC) valida que exista una sesión activa. En el módulo de horarios, se ha implementado un filtrado estricto donde la sesión vincula cada acción (modificación/borrado) al idPersonal correspondiente, garantizando que un usuario no pueda interferir en los logs de otro.

7.2 Prueba con dos usuarios simultáneos

El sistema soporta la concurrencia mediante el uso de un Pool de Conexiones gestionado por la librería mssql. Esto permite que:

1. **Usuario A (Administrador):** Puede estar modificando el horario de un empleado y consultando los logs de auditoría en tiempo real.
2. **Usuario B (Operario):** Al mismo tiempo, puede estar registrando un fichaje o actualizando el estado de una tarea desde otro terminal.

7.3. Gestión de Fallos de Comunicación

Ante una caída de red o falta de respuesta del servidor de base de datos, el sistema implementa una política de Resiliencia Mínima:

- **Detección de Timeout:** Si la conexión no se establece en 15 segundos, el controlador captura la excepción.
- **Feedback al Usuario:** La interfaz de usuario (UI) muestra un "Toast" (notificación emergente) de advertencia y bloquea las acciones de escritura para evitar pérdida de datos en el cliente.

- **Logs de Error:** El sistema registra el fallo en la consola para diagnóstico técnico.

7.4. Evidencia de Operación

- Identificación usuarios

```
[DEBUG] auth:login session sample: {
  IdIdentidad: 94,
  IdUsuario: 36,
  Nombre: 'Máximo Dabarca Maniero (mdbarca)',
  IdTipoUsuario: 1,
  TipoUsuario: 'Personal',
  Departamento: '',
  NumeroGrupos: 3,
  Usuario: 'mdbarca',
  Contraseña: null,
  Obsoleto: 0,
  Color: -983056,
  AvisoBackup: 0,
  Activo: 1,
  ActivoTexto: 'Si',
  AutenticarConActiveDirectory: 0,
  AutenticarConActiveDirectoryTexto: 'No',
  IdTipoValidacionUsuarioDominio: null,
  TipoValidacionUsuarioDominio: null,
  IdDominio: null,
  UsuarioNombre: 'mdbarca//Máximo',
  'ContraseñaAltaSeguridad': 0,
  'CaducidadContraseña': 0,
  'NumeroMesesCaducidadContraseña': 2,
  'FechaCambioContraseña': 2026-03-05T12:08:46.543Z,
  UsuarioNombreSimple: 'Máximo Dabarca Maniero',
  Robot: 0,
  'ContraseñaProvisional': 0,
  'ContraseñaProvisionalTexto': 'No'
}
```

- Usuarios simultáneos

The screenshot displays two overlapping browser windows of a web application. The top window shows a user profile for 'Máximo Dabarca Maniero (mdbarca)' with a status of 'Trabajando — Fichar salida'. The bottom window shows a user profile for 'Miguel Rodríguez Taboas (mrodriguez)' with a status of 'En pausa — Reanudar'. Both windows show a dashboard with task filters and a table of tasks.

Top Window (Máximo Dabarca Maniero):

- Status: Trabajando — Fichar salida
- Filters: Todos 8, Asignada 7, Iniciada 1, Realizada 0, Comprobada 0, En espera 0

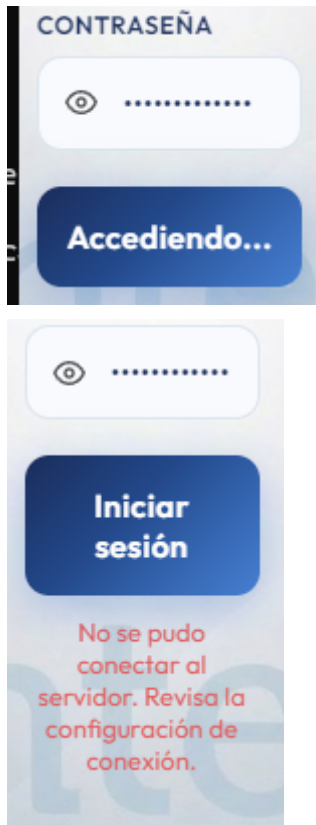
Bottom Window (Miguel Rodríguez Taboas):

- Status: En pausa — Reanudar
- Filters: Todos 9, Asignada 3, Iniciada 6, Realizada 0, Comprobada 0, En espera 0

Table Data (Visible in Bottom Window):

L. RE.	TIPO	CLIENTE	PRESUPUESTO	ALB.	DESCRIPCIÓN
0 h	—	CONFEDERACIÓN COMERCIOS	—	X	bbbbbb

- Fallos de comunicación



8. Despliegue e instalación

8.1 Instalación

La instalación es bastante sencilla porque la aplicación viene empaquetada:

1. Descargar el instalador **TareasGForma-Setup-2.0.0.exe** desde la carpeta *dist/*.
2. Ejecutarlo con permisos de administrador.
3. Seleccionar carpeta de destino (por defecto *C:\Program Files\TareasGForma*).
4. Pulsar Install y esperar.
5. Al terminar, la app aparece en el Menú Inicio y en el escritorio.

8.2 Configuración

Archivo / Variable	Descripción	Valor por defecto	Nota
dist/Config.xml	Configuración de la app (BD, recursos, tema).	<config>...</config>	Editar con cualquier editor XML.

DB_CONNECTION	Cadena de conexión a SQL Server.	Server=localhost;Database=TareasGForma;	Sobrescribe el valor del XML.
APP_MODE	Modo de ejecución.	production	development activa logs detallados.
.env	Parámetros de la base de datos	DB_USER= " " DB_PASSWORD= " "	Es necesario completar estos campos con el nombre de usuario y contraseña de la base de datos.

Para cambiar la configuración de base de datos: abrir dist/Config.xml, modificar los valores de <database> con el host, puerto y credenciales correctos, guardar y reiniciar la aplicación.

8.3 Datos de ejemplo

Para facilitar las pruebas y la evaluación, el proyecto incluye un sistema de carga inicial de datos:

- Script SQL: El archivo **database_creation_script.sql** contiene no solo la estructura, sino también los INSERT iniciales necesarios para:
 - Crear el usuario administrador por defecto (IdIdentidad).
 - Configurar la sociedad de trabajo inicial.
 - Poblar catálogos de tipos de tareas y eventos de presencia.
- Seed Manual: Se recomienda crear al menos dos empleados para probar la independencia de logs y prioridades que el sistema garantiza de forma aislada para cada cuenta.

8.4 Manual de usuario (Breve)

La aplicación se divide en tres módulos funcionales principales:

1. Gestión de Tareas:

- Permite añadir tareas vinculadas a clientes.
- Priorización: El sistema asigna automáticamente la siguiente prioridad disponible. Al modificar o borrar, el sistema recalcula las prioridades para que sean correlativas (1, 2, 3...) sin huecos.

TAREAS ACTIVAS		TAREAS FINALIZADAS												
		Todos: 14 Asignada 13 Iniciada 1 Realizada 0 Comprobada 0 En espera 0												
Nº TAREA +	ESTADO	PRIORIDAD	F. CREACIÓN	F. ENTREGA	INICIADA	FINALIZADA	COMPROBADA	H. EST.	H. RE.	TIPO	CLIENTE	PRESUPUESTO	ALB.	DESCRIPCIÓN
110	Asignada	1	05/06/2026	—	—	—	—	0 h	5 h	—	—	—	X	rtyr
127	Asignada	2	05/06/2026	—	—	—	16/06/2026	0 h	0 h	—	—	—	X	qweweweq
120	Asignada	3	05/06/2026	18/06/2026	18/06/2026	—	—	0 h	0 h	—	—	—	X	sefsfe
109	Asignada	4	05/06/2026	—	—	—	—	0 h	0 h	—	—	—	X	tuutu
107	Asignada	5	05/06/2026	—	—	—	—	0 h	0 h	—	—	—	X	ryy
114	Asignada	6	05/06/2026	—	—	—	—	0 h	0 h	—	—	—	X	68u6u8
112	Asignada	7	05/06/2026	—	—	—	—	0 h	0 h	—	—	—	X	sqwsas
113	Asignada	8	05/06/2026	—	—	—	—	0 h	0 h	—	—	—	X	aw4trw4r
117	Asignada	9	05/06/2026	18/06/2026	—	—	—	0 h	0 h	—	—	—	X	ewfwf
118	Asignada	10	05/06/2026	—	18/06/2026	—	—	0 h	0 h	—	—	—	X	eqdqwd
123	Asignada	11	05/06/2026	—	—	—	—	0 h	0 h	—	—	—	X	wewd
126	Asignada	12	05/06/2026	—	—	—	—	0 h	0 h	—	—	—	X	qwqwq
129	Asignada	13	07/06/2026	—	—	—	—	0 h	0 h	—	—	—	X	asdsad
DESCRIPCIÓN				COMENTARIO					OBSERVACIONES					
rtyr				Añadir comentario...					Añadir observaciones...					

2. Control de Presencia (Horarios):

- Los operarios registran su entrada y salida mediante botones de acción rápida.
- La aplicación captura la IP del dispositivo para auditoría.

Horarios

Trabajador: Máximo Dabarca Maniero (mdbarca)

Refrescar

Fecha

08/06/2026

a

08/06/2026

Buscar

REGISTRO DE HORARIOS

6 registros

FECHA	HORA	EVENTO	COMENTARIO	DISPOSITIVO
08/06/2026	15:11	Salida	-	192.168.56.1
08/06/2026	15:10	Fin pausa	-	192.168.56.1
08/06/2026	15:10	Pausa	-	192.168.56.1
08/06/2026	15:10	Entrada	-	192.168.56.1
08/06/2026	11:11	Salida	-	10.0.0.83
08/06/2026	10:26	Entrada	-	10.0.0.83

3. Panel de Administración y Auditoría:

- Logs de Auditoría:** Al hacer clic en "Ver Logs" en la sección de horarios, el administrador puede ver cada cambio o eliminación realizada.
- Seguridad:** Los logs están filtrados estrictamente por el empleado seleccionado. Las eliminaciones aparecen registradas detallando el motivo, el valor anterior y quién realizó la acción.

Fecha 08/06/2026 a 08/06/2026

Buscar

AUDITORIA DE FICHAJES

4 registros

FECHA/HORA	USUARIO	ACCION	CAMPO	VALOR ANTERIOR	VALOR NUEVO	MOTIVO	ID FICHAJE
08/06/2026 12:00:33	adminTareas	UPDATE	IdControlPresenciaTipoEvento	1	2	ooo	210
08/06/2026 11:59:57	adminTareas	DELETE	Registro	fechaHora=2026-06-08 11:11:00; tipo=4; ip=10.0.0.83	-	ii	209
08/06/2026 11:50:57	adminTareas	UPDATE	IdControlPresenciaTipoEvento	4	1	kmkkk	210
08/06/2026 11:50:57	adminTareas	UPDATE	FechaHora	2026-06-08 11:11:27	2026-06-08 11:11:00	kmkkk	210

9. Pruebas

9.1 Plan de pruebas

El objetivo de este plan es verificar la integridad de los datos, la seguridad en la concurrencia y el correcto funcionamiento de la lógica de negocio automatizada.

¿Qué se va a probar?

- Autenticación y gestión de roles (Admin/User).
- Lógica de priorización automática de tareas (correlatividad y unicidad).
- Registro de presencia y persistencia de fichajes.
- Sistema de auditoría (logs de modificaciones y eliminaciones).
- Aislamiento de datos entre cuentas de empleados.

9.2 Casos de prueba

ID	Caso de Prueba	Objetivo	Resultado Esperado
CP01	Login de usuario	Validar acceso	Acceso concedido y carga de sesión correcta.
CP02	Creación de tareas	Probar prioridad auto	La nueva tarea toma la prioridad MAX + 1.
CP03	Finalización de tareas	Validar limpieza	Al poner fecha de fin, la prioridad queda vacía (NULL).

CP04	Registro de Fichajes	Validar presencia	Se guarda el evento con fecha/hora e IP del equipo.
CP05	Modificación de horario	Probar auditoría	Se genera un log tipo UPDATE con el valor anterior y nuevo.
CP06	Eliminación de horario	Probar persistencia	El registro desaparece pero se crea un log tipo DELETE con el motivo.
CP07	Aislamiento de logs	Validar privacidad	Los logs de "Empleado A" no son visibles al consultar al "Empleado B".
CP08	Recálculo de prioridades	Probar reordenación	Al borrar la tarea 1, la tarea 2 pasa a ser la 1 automáticamente.

10. Resultados, conclusiones y vías futuras

10.1 Qué conseguí

1. Tener una aplicación de escritorio funcional de verdad:

- Cumplimiento: Se ha desarrollado una aplicación nativa utilizando el ecosistema Electrón JS, lo que permite una ejecución fluida en sistemas Windows. La aplicación es totalmente operativa, permitiendo la gestión integral de datos sin depender de un navegador web externo.

2. Centralizar la información en una base de datos relacional:

- Cumplimiento: Se ha migrado toda la lógica de almacenamiento a SQL Server. Mediante el uso de un pool de conexiones gestionado por el servidor, se ha logrado una gestión centralizada que garantiza la consistencia de los datos y permite el acceso concurrente de múltiples usuarios.

3. Autenticación de usuarios con control de sesión:

- Cumplimiento: Se ha implementado una pantalla de login vinculada a la tabla de seguridad de la base de datos. El sistema diferencia roles (Administrador/Usuario) y mantiene una sesión activa que protege los canales de comunicación (IPC), asegurando que solo los usuarios autorizados realicen cambios.

4. CRUD completo para clientes, tareas, contactos y presupuestos:

- Cumplimiento: Se han desarrollado modelos y controladores específicos para estas cuatro entidades. Cada una cuenta con una interfaz que permite crear, leer, actualizar y borrar registros, aplicando validaciones de integridad referencial para evitar errores en la base de datos.

5. Módulo de tiempos y de presencia:

- Cumplimiento: Se ha creado un panel interactivo de fichaje en tiempo real. Este objetivo se ha superado con la implementación de un sistema de auditoría (logs) que

registra modificaciones y eliminaciones, asociándolas de forma segura a cada empleado mediante una columna de IdPersonal específica en los registros de auditoría.

6. Interfaz navegable y usable:

- Cumplimiento: Se ha aplicado un diseño centrado en el usuario, utilizando CSS moderno (Flexbox/Grid) y tipografías legibles. La navegación es intuitiva gracias a un sistema de pestañas y modales, y se han incluido notificaciones de estado ("toasts") para mejorar la experiencia de uso.

7. Configuración de conexión sin credenciales hardcodeadas:

- Cumplimiento: Se ha integrado la librería dotenv, permitiendo que toda la configuración sensible (usuario, contraseña y servidor de BD) se almacene en un archivo externo .env. Esto cumple con los estándares de seguridad y facilita el despliegue en diferentes entornos.

Logros Adicionales (Valor Añadido)

Además de los objetivos previstos, se han alcanzado metas técnicas avanzadas:

- **Gestión de Prioridades Automática:** El sistema reordena las prioridades de las tareas de forma inteligente para evitar duplicados y huecos.
- **Independencia de Logs:** Se ha blindado el sistema de auditoría para que los registros de un empleado sean invisibles para otros, garantizando la privacidad en un entorno multiusuario.
- **Estructura Modular:** El código se ha organizado en una arquitectura de capas (Modelos, Controladores, Vistas e IPC), lo que facilita enormemente el mantenimiento y escalabilidad futura del software.

10.2 Conclusiones + lecciones aprendidas

El desarrollo de TareasGForma ha supuesto un reto técnico que ha permitido consolidar conocimientos avanzados en el desarrollo de software de escritorio moderno. A continuación, se detallan las principales lecciones extraídas de cada área tecnológica:

Sobre Electron JS

He aprendido que Electrón no es simplemente un "navegador que envuelve una web", sino un ecosistema complejo que requiere gestionar dos mundos distintos: el Proceso Principal (Node.js) y el Proceso de Renderizado (Navegador). La principal lección ha sido comprender que la lógica pesada y el acceso a datos deben vivir siempre en el proceso principal para mantener una interfaz fluida y segura.

Sobre SQL Server

El trabajo con SQL Server me ha enseñado la importancia de la integridad de los datos. He aprendido a delegar lógica compleja al motor de base de datos mediante Procedimientos Almacenados y funciones de T-SQL (como el uso de ROW_NUMBER() para el recálculo de prioridades). También he comprendido la importancia de gestionar correctamente los valores NULL y los connection pools para evitar fugas de memoria y bloqueos en entornos concurrentes.

Sobre la Comunicación IPC

Esta ha sido la lección más crítica sobre arquitectura. He aprendido que los canales IPC son el "pegamento" de la aplicación. Una buena estructura de canales (usando `ipcMain.handle` e `ipcRenderer.invoke`) permite una comunicación asíncrona que evita que la aplicación se congele. Además, he aprendido a usar el `preload.js` como un puente seguro, exponiendo solo las funciones estrictamente necesarias.

Sobre la Arquitectura por Capas

Al principio, la tentación de escribir código directamente en la vista era alta, pero he aprendido que la separación de responsabilidades (Modelos, Controladores y Vistas) es lo que hace que un proyecto sea mantenible. Gracias a esta arquitectura, cuando tuvimos que implementar el sistema de auditoría o cambiar la lógica de prioridades, pudimos hacerlo de forma quirúrgica sin romper el resto de la aplicación.

Sobre la Seguridad y el Aislamiento

He comprendido que la seguridad no termina en el login. La implementación de los Logs de Auditoría me ha enseñado que es vital registrar el "quién, cuándo y por qué" de cada acción administrativa. Especialmente instructivo fue el reto de garantizar el aislamiento de datos entre empleados; aprendí que el servidor debe validar siempre la identidad del usuario en cada petición para evitar que un operario acceda accidentalmente a la información privada de otro.

En conclusión, este proyecto me ha enseñado que el desarrollo de software profesional no se trata solo de escribir código que funcione, sino de diseñar sistemas mantenibles, seguros y escalables. La transición de una aplicación sencilla a un sistema multiusuario con auditoría ha sido la experiencia de aprendizaje más valiosa del proceso.

10.3 Por dónde seguir

1. Añadir más validaciones en formularios, especialmente en los de presencia.
2. Mejorar la exportación de informes: más formatos, mejor visualización.
3. Un panel de estadísticas o métricas básicas.
4. Perfiles de usuario con permisos más granulares.
5. Si el proyecto escala, separar la lógica en un backend desacoplado.
6. Mejorar el sistema de notificaciones internas.

10.4 Reflexión Personal

El desarrollo de Tareas GForma ha sido un desafío técnico que me ha permitido consolidar conocimientos avanzados en el desarrollo de software de escritorio. La parte más compleja del proyecto fue, sin duda, garantizar la integridad de los datos en un entorno multiusuario concurrente. Asegurar que la lógica de priorización automática de tareas funcionara correctamente cuando varios usuarios operaban simultáneamente, sin provocar conflictos ni duplicados en la base de datos, me obligó a profundizar en el uso de transacciones SQL y a optimizar la comunicación entre el proceso de renderizado y el proceso principal de Electrón.

De este desarrollo he aprendido tres lecciones fundamentales:

> **La importancia de la arquitectura:** La separación clara de responsabilidades (Modelos, Controladores y Vistas) no es una sugerencia, sino una necesidad vital para que un proyecto sea escalable y mantenible.

> **El poder del backend en SQL:** He comprendido que es mucho más seguro y eficiente delegar la lógica de negocio compleja (como el cálculo de prioridades o auditorías) al motor de base de datos mediante procedimientos almacenados, en lugar de intentar gestionarla desde el lado del cliente.

> **La comunicación IPC como eje:** Aprendí que gestionar correctamente los canales IPC es el "pegamento" de Electrón; una buena estructura de comunicación asíncrona es lo que diferencia una aplicación fluida de una que se congela bajo carga.

10.5 Métricas del proyecto

Categoría	Métrica	Cantidad	Descripción
Base de datos	Tablas SQL	9	Incluye entidades principales, seguridad y logs de auditoría.
Arquitectura	Modelos(Data Access)	6	Clases encargadas de la persistencia y consultas SQL.
Arquitectura	Controladores	5	Clases que gestionan la lógica de negocio y validaciones.
Interfaz	Pantallas/ Vistas	8	Ventanas independientes (Tareas, Horarios, Logs, Login, etc.).
Comunicación	Canales IPC	~45	Puntos de comunicación entre el proceso Main y Renderer.
Calidad	Casos de prueba	8	Pruebas funcionales y de integración documentadas.
Usuarios	Roles de usuarios	2	Administrador y Usuario Estándar con permisos diferenciados.
Código	Líneas de código	~7.500	Suma de JavaScript, HTML, CSS y Scripts de Base de Datos.
Funcionalidades	Módulos Principales	7	Tareas, Clientes, Presupuestos, Presencia, Auditoría, Informes y Configuración.

10.6 Tabla resumen

Entidad	Descripción	Cantidad
Usuarios	Personal registrado en el sistema	29
Tareas	Total de tareas creadas	17
Fichajes	Total de registros de presencia	153
Logs	Entradas de auditoría (modificaciones/borrados)	69

```
1 use GestionFormacion
2 SELECT COUNT(*) FROM dbo.gf_Personal;
3 SELECT COUNT(*) FROM dbo.Tareas; -- Para Tareas
4 SELECT COUNT(*) FROM dbo.ControlPresencia_Fichajes; -- Para Fichajes
5 SELECT COUNT(*) FROM dbo.ControlPresencia_Fichajes_Logs; -- Para Logs
6
```

00 % No se encontraron problemas.

Resultados Mensajes

	(Sin nombre de columna)
1	29

	(Sin nombre de columna)
1	17

	(Sin nombre de columna)
1	153

	(Sin nombre de columna)
1	69

11. Glosario

Electron: framework que permite crear aplicaciones de escritorio usando tecnologías web (HTML, CSS, JS), empaquetándolas con [Node.js](#).

IPC (Inter-Process Communication): mecanismo de Electrón para comunicar de forma segura el proceso de renderizado (UI) con el proceso principal (Main Process).

Preload: script de Electron que actúa de puente entre la interfaz y Node.js, exponiendo solo las funciones necesarias.

DAO (Data Access Object): patrón de diseño que encapsula las consultas a la base de datos, separándolas de la lógica de negocio.

Pool de conexiones: conjunto de conexiones a la base de datos reutilizadas para mejorar el rendimiento y soportar acceso concurrente.

mssql: paquete Node.js para conectar con SQL Server y ejecutar consultas T-SQL.

CRUD: acrónimo de Create, Read, Update, Delete — operaciones básicas sobre una base de datos.

T-SQL: dialecto de SQL propio de Microsoft SQL Server.

Toast: notificación emergente no intrusiva que aparece brevemente en la interfaz para informar al usuario de un estado o error.

Seed / datos semilla: datos iniciales insertados en la base de datos para facilitar pruebas o el arranque del sistema.

12. Bibliografía

Electron JS (v25.x+): Documentación oficial para la creación de aplicaciones de escritorio multiplataforma con tecnologías web.

* Fuente: <https://www.electronjs.org/docs>

Node.js (v18.x+): Entorno de ejecución de JavaScript en el servidor.

* Fuente: <https://nodejs.org/en/docs/>

Microsoft SQL Server: Manual de referencia para T-SQL y administración de bases de datos relacionales.

* Fuente: <https://learn.microsoft.com/es-es/sql/sql-server>

mssql (node-mssql): Driver de conexión para la comunicación entre Node.js y SQL Server.

* Uso: Gestión de pools de conexiones y ejecución de procedimientos almacenados.

* Referencia: <https://www.npmjs.com/package/mssql>

dotenv: Gestión de variables de entorno para la protección de credenciales de base de datos.

* Referencia: <https://www.npmjs.com/package/dotenv>

Claude.ia:

* Referencia: <https://claude.ai/>

electron-builder: Herramienta para el empaquetado y distribución de la aplicación (generación de ejecutables).

* Referencia: <https://www.electron.build/>

pdf-lib / jsPDF: Librería para la generación dinámica de informes de presencia y logs en formato PDF.

* Referencia: <https://pdf-lib.js.org/>

Visual Studio Code: Entorno de desarrollo integrado (IDE).

SQL Server Management Studio (SSMS): Gestión y diseño del esquema de base de datos.

Mermaid.js: Generación de diagramas UML de clases y secuencia integrados en la documentación.

* Fuente: <https://mermaid.js.org/>

ANEXOS

Anexo A – Capturas adicionales

- Pantallas principales (5.3)

- Pantalla conexión

Conexión
Selecciona o añade una conexión al servidor

CONEXIÓN

proyecto.intecoingenieria.com,12421

Base de datos

GestionFormacion

Probar conexión

AÑADIR CONEXIÓN

Servidor Puerto

ej: mi-servidor-sql 1433

+ Añadir

Guardar Salir

www.intecoingenieria.com · Tel: 986 493 459

- Pantalla Login

nexova
soluciones de gestión empresarial

Bienvenido
Accede a tu cuenta para continuar

USUARIO

mdbarca

CONTRASEÑA

.....

Iniciar sesión

Configurar conexión

- Pantalla dashboard tareas

General

Ver

Trabajando — Fichar salida

Máximo Dabarca Maniero (Admin)

Mé Purtil

+🔍📄⬆️⬇️🗑️🔄🕒📅📄📄

TAREAS ACTIVAS

TAREAS FINALIZADAS

Todos: 8

Asignada 7

Iniciada 1

Realizada 0

Comprobada 0

En espera 0

Nº TAREA	ESTADO	PRIORIDAD	F. CREACIÓN	F. ENTREGA	INICIADA	FINALIZADA	COMPROBADA	H. EST.	H. RE.	TIPO	CLIENTE	PRESUPUESTO	ALB.	DESCRIPCIÓN
114	Asignada	1	05/06/2026	—	—	—	—	0 h	0 h	—	—	—	X	68u6u8
112	Asignada	2	05/06/2026	—	—	—	—	0 h	0 h	—	—	—	X	sqwss
113	Asignada	3	05/06/2026	—	—	—	—	0 h	0 h	—	—	—	X	aw4trwdr
117	Asignada	4	05/06/2026	18/06/2026	—	—	—	0 h	0 h	—	—	—	X	ewfuf
118	Asignada	5	05/06/2026	—	18/06/2026	—	—	0 h	0 h	—	—	—	X	eqdqwd
123	Asignada	6	05/06/2026	—	—	—	—	0 h	0 h	—	—	—	X	wewd
125	Asignada	7	05/06/2026	—	—	—	—	0 h	0 h	—	—	—	X	qwewq
132	Iniciada	8	07/06/2026	—	06/06/2026	—	—	0 h	0 h	—	—	—	X	888

- Pantalla modificar / crear tarea:

+ Nueva tarea

DATOS DE LA TAREA

FECHA DE CREACIÓN

08/06/2026

F. ENTREGA

dd/mm/aaaa

F. INICIO

dd/mm/aaaa

F. FIN

dd/mm/aaaa

F. COMPROBACIÓN

dd/mm/aaaa

RESPONSABLE 1 (quien manda la tarea)

— Seleccionar responsable —

RESPONSABLE 2 (a quien va la tarea)

☒ Activo
Máximo Dabarca Maniero

TIPO DE TAREA

Diaria

F. EN ESPERA

dd/mm/aaaa

CARPETA DE TRABAJO

Ruta...

CONTENIDO

DESCRIPCIÓN

Descripción de la tarea...

COMENTARIO

Comentario...

OBSERVACIONES

Observaciones...

CLIENTE Y PRESUPUESTO

CLIENTE

Cliente...

Q

☐ Cobrar al cliente
☒ Seleccionar presupuesto

Salir

Guardar

OBSERVACIONES

Observaciones...

CLIENTE Y PRESUPUESTO

CLIENTE

Cliente...

Q

☐ Cobrar al cliente
☒ Seleccionar presupuesto

PRESUPUESTO

Presupuesto...

Q

LÍNEA DE PRESUPUESTO

Línea...

CONTACTO

Contacto...

Q

HORAS ESTIMADAS

0

HORAS REALES

0,00

DEPARTAMENTO

Departamento...

ASIGNAR TIEMPOS

COMENTARIO

Descripción del trabajo realizado...

FECHA

08/06/2026

HORAS

Registrar tiempo

Salir

Guardar

- Pantalla Ver tiempos:

Ver tiempos

Tarea: 55

Refrescar

Asignada a

Máximo Dabarca Maniero

Fecha del tiempo

dd/mm/aaaa

a

dd/mm/aaaa

Buscar

+

REGISTROS DE TIEMPOS

1 registros

ASIGNADA A	FECHA	H. REALES	H. ESTIM.	NO TAREA	PRESUPUESTO	CLIENTE	DESCRIPCION DE LA TAREA	COMENTARIO DE LA TAREA
Máximo Dabarca Maniero	03/06/2026	2.00 h						
Máximo Dabarca Maniero		2.00 h	0.00 h	55	—	—	drdt	

Filtrando: Tarea 55 · Asignada a Máximo Dabarca Maniero

- Pantalla Horarios:

Horarios

Trabajador: Máximo Dabarca Maniero (mdbarca)

Refrescar

Fecha

04/06/2026

a

04/06/2026

Buscar

REGISTRO DE HORARIOS					3 registros
FECHA	HORA	EVENTO	COMENTARIO	DISPOSITIVO	
04/06/2026	02:32	Fin pausa	-	192.168.56.1	
04/06/2026	00:39	Pausa	-	192.168.56.1	
04/06/2026	00:25	Entrada	-	192.168.56.1	

Filtrando: 2026-06-04 a 2026-06-04

- Pantalla Informe presencia:

Informe presencia

Trabajador: Máximo Dabarca Maniero (mdbarca)

Refrescar

Fecha

04/06/2026

a

04/06/2026

Buscar

Generar PDF

Exportar CSV

RESUMEN			2 registros
ENTRADAS	SALIDAS	TIEMPO	
04/06/2026 00:25 ● Entrada	04/06/2026 00:39 ● Pausa	0 h 14 min	
04/06/2026 02:32 ● Fin pausa	-	0 h 00 min	
Resumen 2026-06-04		0 h 13 min	

Filtrando: 2026-06-04 a 2026-06-04

- Más pantallas
- Pantalla principal + filtros de búsqueda

GeneralVer

Trabajando — Fichar salida

Máximo Dabarca Maniero (Admin)

Mé Perfil

- Pantalla ver tarea (solo lectura)

Ver tarea

DATOS DE LA TAREA

FECHA DE CREACIÓN: 05/06/2026

F. ENTREGA: dd/mm/aaaa

F. INICIO: dd/mm/aaaa

F. FIN: dd/mm/aaaa

F. COMPROBACIÓN: dd/mm/aaaa

RESPONSABLE 1 (quien manda la tarea): Máximo Dabarca Maniero

RESPONSABLE 2 (a quien va la tarea): Máximo Dabarca Maniero

TIPO DE TAREA:

F. EN ESPERA: dd/mm/aaaa

CARPETA DE TRABAJO: Ruta...

CONTENIDO

DESCRIPCIÓN: 68iu6iu8

COMENTARIO: Comentario...

OBSERVACIONES: Observaciones...

CLIENTE Y PRESUPUESTO

CLIENTE: Cliente...

PRESUPUESTO: Presupuesto...

LÍNEA DE PRESUPUESTO: Línea...

Salir

- Pantalla Ver Logs (solo admin):

Logs de horarios

Logs de horarios

Trabajador: adminTareas

Refrescar

Generar PDF

Fecha

08/06/2026

a

08/06/2026

Buscar

AUDITORIA DE FICHAJES

4 registros

FECHA/HORA	USUARIO	ACCION	CAMPO	VALOR ANTERIOR	VALOR NUEVO	MOTIVO	ID FICHAJE
08/06/2026 12:00:33	adminTareas	UPDATE	IdControlPresenciaTipoEvento	1	2	ooo	210
08/06/2026 11:59:57	adminTareas	DELETE	Registro	fechaHora=2026-06-08 11:11:00; tipo=4; ip=10.0.0.83	-	ii	209
08/06/2026 11:50:57	adminTareas	UPDATE	IdControlPresenciaTipoEvento	4	1	kmkkk	210
08/06/2026 11:50:57	adminTareas	UPDATE	FechaHora	2026-06-08 11:11:27	2026-06-08 11:11:00	kmkkk	210

- Pantalla Clientes:

TareasGForma

Nombre

Pais

Email

Nº Doc.

Tipo doc.

Número

Teléfono

Provincia

Buscar

Asignar

NUMERO	N. COMERCIAL	R. SOCIAL	TIPO DOC.	Nº DOC.	PAÍS
26	NULL	INTECO INGENIERIA AVANZADA S.L.	CIF	836856540	España
118	NULL	NULL	DNI	32894770D	NULL
147	NULL	NULL	Pasaporte	123456789	NULL
148	NULL	NULL	Pasaporte	123456789	NULL
946	NULL	TRANSPORTES Y EXCAVACIONES AGUAS NUEVAS SCL	CIF	F02309193	España
1013	NULL	AGENCIA CREATIVA TOUCHE SL	CIF	b47762851	España
1042	NULL	NULL	Pasaporte	NULL	España
1043	NULL	NULL	Pasaporte	NULL	España
1044	NULL	NULL	Pasaporte	NULL	España
1045	NULL	NULL	Pasaporte	NULL	España
1046	NULL	NULL	Pasaporte	NULL	España
1047	NULL	NULL	Pasaporte	NULL	España
1048	NULL	NULL	Pasaporte	NULL	España

Filtrando: Sociedad

BD: GestionFormacion proyecto.intecoingeniería.com,12421

Registros: 0 - Tiempo: 0 s

- Pantalla Presupuestos:

Presupuestos

Cliente

INTECO INGENIERIA AVANZADA S.L.

Año

Con f. entrega

Sin seleccionar

Número

Mes/Trimestre

Albaraneado

Sin seleccionar

Fecha

Vers.

Facturado

Sin seleccionar

Descripción

Tipo

Estado

Sin seleccionar

Est. produc.

Descr. línea

Preparación

Enviado

Buscar

Asignar

NÚMERO	VERSIÓN	FECHA	CLIENTE	DESCRIPCIÓN	F. ENTREGA	IMPORTE €	ESTADO	FECHA ESTADO	EST. PRODUCCIÓN
• 2603-005	0	03/06/2026	INTECO INGENIERIA AVANZADA S.L.	Probando presupuestos DUPLICADO	—	4,90	Preparación	—	Pendiente producción
• 2603-004	0	03/06/2026	INTECO INGENIERIA AVANZADA S.L.	Probando presupuestos	—	4,90	Preparación	—	Albaraneado

LÍNEAS DE PRESUPUESTO

Nº ORDEN	CÓD. ART.	DESCRIPCIÓN	CANTIDAD	UN. MEDIDA	PRECIO €	DESC. %	IMPORTE €	REQ. TAREA	REQ. PEDIDO	CARPETA DE TRABAJO	PROVEEDOR	ALBARANES	Nº TAREAS
----------	-----------	-------------	----------	------------	----------	---------	-----------	------------	-------------	--------------------	-----------	-----------	-----------

Seleccione un presupuesto para ver sus líneas.

Filtrando: Cliente contiene "INTECO INGENIERIA AVANZADA S.L."

- Pantalla Contactos:

TareasGForma

FILTROS GENERALES

Cliente

INTECO INGENIERIA AVANZADA S.L.

Tipo contacto

☐ Reclamada

Personal

Máximo Dabarca Maniero

Fec. contacto

08/06/2026

a

08/06/2026

Estado contacto

Clasificado como

Buscar

PERSONAL	FECHA	HORA	NUM.	PROPIETARIO	DIRECCIÓN	PROVINCIA	POBLACIÓN	Nº TELÉFONO	PERSONA DE CONTACTO	TIPO	PLANIFICADO/REALIZADO	RELL.	OBSERVACIONES
Sin resultados. Use los filtros para buscar contactos.													

Anexo B – Evidencias de pruebas

- Casos de pruebas (9.2)

CP01

```
]
[DEBUG] auth:login session sample: {
  IdIdentidad: 94,
  IdUsuario: 36,
  Nombre: 'Máximo Dabarca Maniero (mdbarca)',
  IdTipoUsuario: 1,
  TipoUsuario: 'Personal',
  Departamento: '',
  NumeroGrupos: 3,
  Usuario: 'mdbarca',
  Contraseña: null,
  Obsoleto: 0,
  Color: -983056,
  AvisoBackup: 0,
  Activo: 1,
  ActivoTexto: 'Si',
  AutenticarConActiveDirectory: 0,
  AutenticarConActiveDirectoryTexto: 'No',
  IdTipoValidacionUsuarioDominio: null,
  TipoValidacionUsuarioDominio: null,
  IdDominio: null,
  UsuarioNombre: 'mdbarca//Máximo',
  'ContraseñaAltaSeguridad': 0,
  'CaducidadContraseña': 0,
  'NumeroMesesCaducidadContraseña': 2,
  'FechaCambioContraseña': 2026-03-05T12:08:46.543Z,
  UsuarioNombreSimple: 'Máximo Dabarca Maniero',
  Robot: 0,
  'ContraseñaProvisional': 0,
  'ContraseñaProvisionalTexto': 'No'
}
[WARN] resolveIdPersonal: matched by name/usuario Æ IdPersonal: 36
[DEBUG] auth:login IdIdentidad: 94 Æ IdPersonal: 36
[DEBUG] IdUsuario sesi|n: 36
[DEBUG] IdIdentidad sesi|n: 94
[DEBUG] tareas:getAll filtros { idPersonalAsignado: 36 }
[WARN] up_bp_Tareas_Select devolvi| 0 registros. Usando fallback directo de Tareas (filtrado).
[DEBUG] tareas:getAll count 10
```

CP02

General Ver Trabajando — Fichar salida Máximo Dabarca Maniero (mdbarca) Mi Perfil

+ 🔍 ↺ ↑ ↓ 🗑️ 📄 🔄 ⌛ 📅 ▼

TAREAS ACTIVAS TAREAS FINALIZADAS

Todos 2 Asignada 2 Iniciada 0 Realizada 0 Comprobada 0 En espera 0

Nº TAREA ↑	ESTADO	PRIORIDAD	F. CREACIÓN	F. ENTREGA	INICIADA	FINALIZADA	COMPROBADA	H. EST.	H. RE.	TIPO	CLIENTE	PRESUPUESTO	ALB.	DESCRIPCIÓN
112	Asignada	1	05/06/2026	—	—	—	—	0 h	0 h	—	—	—	✗	sqwsas
113	Asignada	2	05/06/2026	—	—	—	—	0 h	0 h	—	—	—	✗	aw4trw4r

Fecha

08/06/2026

a

08/06/2026

Buscar

AUDITORIA DE FICHAJES

7 registros

FECHA/HORA	USUARIO	ACCION	CAMPO	VALOR ANTERIOR	VALOR NUEVO	MOTIVO	ID FICHAJE
08/06/2026 18:31:32	adminTareas	UPDATE	IdControlPresenciaTipoEvento	2	1	prueba update	222
08/06/2026 18:31:32	adminTareas	UPDATE	FechaHora	2026-06-08 16:40:25	2026-06-08 16:40:00	prueba update	222
08/06/2026 18:30:13	adminTareas	DELETE	Registro	fechaHora=2026-06-08 17:11:39; tipo=I; ip=192.168.56.1	-	prueba	224

CP07

Logs de horarios

Logs de horarios

Trabajador: adminTareas

Refrescar

Generar PDF

Fecha

08/06/2026

a

08/06/2026

Buscar

AUDITORIA DE FICHAJES

7 registros

FECHA/HORA	USUARIO	ACCION	CAMPO	VALOR ANTERIOR	VALOR NUEVO	MOTIVO	ID FICHAJE	ID LOG
08/06/2026 18:31:32	adminTareas	UPDATE	IdControlPresenciaTipoEvento	2	1	prueba update	222	69
08/06/2026 18:31:32	adminTareas	UPDATE	FechaHora	2026-06-08 16:40:25	2026-06-08 16:40:00	prueba update	222	68
08/06/2026 18:30:13	adminTareas	DELETE	Registro	fechaHora=2026-06-08 17:11:39; tipo=I; ip=192.168.56.1	-	prueba	224	67
08/06/2026 12:00:33	adminTareas	UPDATE	IdControlPresenciaTipoEvento	1	2	ooo	210	64

46.26.18.183:15217 - Conexión a Escritorio remoto

Logs de horarios

Logs de horarios

Trabajador: adminTareas

Fecha

08-06-2026

a

08-06-2026

AUDITORIA DE FICHAJES

FECHA/HORA	USUARIO	ACCION	CAMPO	VALOR ANTERIOR	VALOR NUEVO	MOTIVO
08/06/2026 12:01:07	adminTareas	UPDATE	IdControlPresenciaTipoEvento	3	1	oooooooo
08/06/2026 12:01:07	adminTareas	UPDATE	FechaHora	2026-06-08 11:51:39	2026-06-08 11:51:00	oooooooo
08/06/2026 11:51:51	adminTareas	UPDATE	IdControlPresenciaTipoEvento	4	2	yyy
08/06/2026 11:51:51	adminTareas	UPDATE	FechaHora	2026-06-08 11:51:41	2026-06-08 11:51:00	yyy

CP08

General

Ver

Trabajando — Fichar salida

Máximo Dabarca Maniero (mdbarca)

Mi Perfil

<

TAREAS ACTIVAS

TAREAS FINALIZADAS

Todos: 2Asignada 2Iniciada 0Realizada 0Comprobada 0En espera 0

Nº TAREA ↑	ESTADO	PRIORIDAD	F. CREACIÓN	F. ENTREGA	INICIADA	FINALIZADA	COMPROBADA	H. EST.	H. RE.	TIPO	CLIENTE	PRESUPUESTO	ALB.	DESCRIPCIÓN
112	Asignada	1	05/06/2026	—	—	—	—	0 h	0 h	—	—	—	X	sqwsas
113	Asignada	2	05/06/2026	—	—	—	—	0 h	0 h	—	—	—	X	aw4trw4r